

Introdução às Redes Neurais

Paulo C. Marques F.

Insper

Nestas aulas iremos tratar dos conceitos envolvidos nas redes neurais artificiais, que formam a base de um estilo de aprendizagem de máquina muito importante para a nossa época, denominado *Deep Learning*.

Iremos desenvolver as ideias desde o princípio, indicando algumas etapas do seu desenvolvimento histórico.

O conceito matemático de rede neural artificial surge como analogia às redes neurais biológicas no trabalho de McCulloch e Pitts em 1943.

A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY

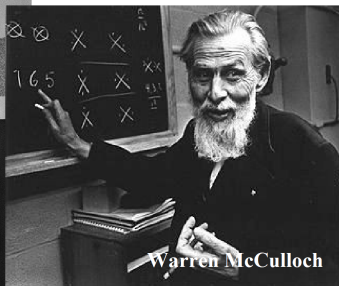
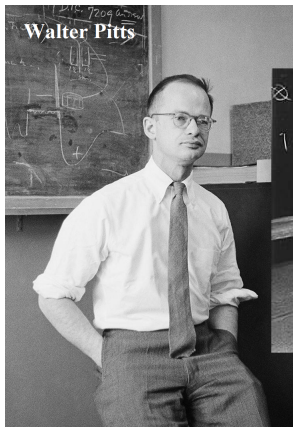
- WARREN S. MCCULLOCH AND WALTER PITTS
University of Illinois, College of Medicine,
Department of Psychiatry at the Illinois Neuropsychiatric Institute,
University of Chicago, Chicago, U.S.A.

Trata-se de um trabalho bastante abstrato, desenvolvido ao estilo da Lógica Matemática, cuja leitura em nossa época torna-se bastante desafiadora. De fato, para um leitor moderno é difícil identificar neste trabalho os elementos atuais do conceito de uma rede neural artificial. Nota-se, em termos atuais, a presença neste trabalho de uma função de ativação estilo “degrau”, que não é utilizada nos modelos contemporâneos. Note-se que neste paper surge um mecanismo de “memória”, predecessor do processo de formação de “memórias” em redes neurais recorrentes modernas.

Veremos ao longo do nosso desenvolvimento que a analogia biológica é limitada, no sentido de que as técnicas atuais, que levaram a soluções de alta performance preditiva, envolvem processos que dificilmente ocorrem no cérebro humano. O papel da analogia biológica parece ser o de guiar e inspirar, sem determinar a forma específica das soluções encontradas. Doravante, o termo *rede neural* será utilizado para denominar o modelo matemático de uma rede neural artificial. Quando precisarmos fazer referência a uma rede neural biológica, o faremos de maneira explícita.

Detalhes dramáticos da curta e conturbada vida de Walter Pitts e de seu trabalho com Warren McCulloch são narrados neste artigo da revista Nautilus.

<https://bit.ly/3L8Sv6C>

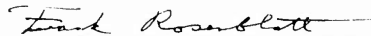


Um desenvolvimento muito mais conectado às soluções atuais foi o modelo *perceptron*, criado por Frank Rosenblatt nos anos 1960.

PRINCIPLES OF NEURODYNAMICS

PERCEPTRONS AND THE THEORY OF BRAIN MECHANISMS

BY:



Frank Rosenblatt

*Director, Cognitive Systems Research Program
Cornell University, Ithaca, N. Y.*

15 MARCH 1961

Assista a este filme da época sobre o desenvolvimento do *perceptron*.

<https://bit.ly/3AxK2Fc>



Minsky e Papert demonstraram em 1969 no livro “Perceptrons” que o modelo *perceptron* original de Rosenblatt possuía limitações intrínsecas, no sentido de que seria impossível aprender certas funções simples (como a função lógica XOR) usando um *perceptron*. De fato, é possível provar que o algoritmo de aprendizagem do *perceptron* converge apenas quando os dados são linearmente separáveis.

Este resultado foi mal compreendido à época e levou a uma perda geral de interesse no *perceptron*, este vovô das redes neurais modernas. Isso tudo a despeito do fato de que no próprio livro de Minsky e Papert é apontado o caminho para um *perceptron* mais geral, multicamadas, que não possui as limitações mencionadas acima, e que é essencialmente o que denominamos hoje em dia por uma rede neural *feed-forward* (ou *multilayer perceptron*). No entanto, vale notar que Minsky e Papert não propuseram um método geral para o treinamento desta extensão do *perceptron* (o que viria a acontecer apenas nos anos 1980).

A intuição por detrás das redes neurais reside no fato de que podemos escrever uma função arbitrariamente “complicada” compondo funções bastante simples.

Por exemplo, a função $f(x) = \sqrt{1 + e^{x^2}}$ pode ser escrita compondo-se as funções

$$g_1(x) = x^2 \quad g_2(x) = e^x \quad g_3(x) = 1 + x \quad g_4(x) = \sqrt{x},$$

da seguinte maneira (verifique):

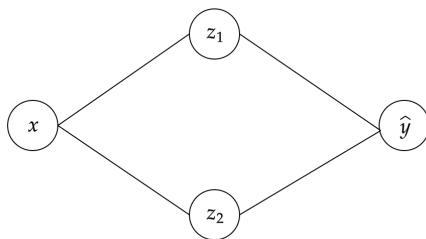
$$f(x) = (g_4 \circ g_3 \circ g_2 \circ g_1)(x).$$

Em uma rede neural, as funções “elementares” terão parâmetros que aprenderemos de maneira supervisionada. Em termos da analogia biológica, cada parâmetro regularia a intensidade da conexão de uma “sinapse”. O cérebro humano tem por volta de 600 trilhões de sinapses.

Nossa estratégia será começar com um problema de regressão muito simples, com apenas uma variável preditora.

Utilizaremos uma arquitetura de rede neural simplificada que nos permitirá entender diversos conceitos fundamentais: arquitetura, camada de entrada, camadas internas e suas unidades (ou neurônios), camada de saída, pesos e vieses, função de ativação, função de perda, gradiente descendente, *backpropagation*, aprendizagem *on-line*, épocas, gradiente descendente estocástico, mini-lotes, regularização, parada prematura e *dropout*.

No *diagrama neuronal* abaixo temos uma unidade (x) na camada de entrada, duas unidades/neurônios (z_1 e z_2) na camada interna e uma unidade ($\hat{y}$) na camada de saída.



A ideia geral é que cada unidade na camada interna faz uma regressão linear na variável preditora da camada de entrada e aplica uma função não-linear ao resultado, denominada *função de ativação*.

Neste exemplo, usaremos uma função de ativação logística, definida por $\sigma(u) = 1/(1 + e^{-u})$. Verifique que $\sigma'(u) = \sigma(u)(1 - \sigma(u))$.

De maneira explícita, temos que:

$$z_1 = \sigma(w_1x + b_1), \quad z_2 = \sigma(w_2x + b_2).$$

Neste problema de regressão, a unidade na camada de saída faz apenas uma regressão linear nas duas unidades da camada interna.

$$\hat{y} = w_3z_1 + w_4z_2 + b_3$$

Podemos escrever a função resultante na forma extensa:

$$\hat{y} = w_3\sigma(w_1x + b_1) + w_4\sigma(w_2x + b_2) + b_3$$

Os w_i 's são denominados “pesos” e os b_i 's são denominados “vieses”. O processo de calcular $\hat{y}$ a partir de x , para determinados valores dos pesos e vieses, é denominado *forward propagation*.

Temos em mãos um conjunto de dados de treinamento $\{(x_i, y_i)\}_{i=1}^n$.

Nosso objetivo é aprender os pesos e vieses da rede neural, denotados como um único vetor

$$\theta = (w_1, b_1, w_2, b_2, w_3, w_4, b_3) \in \mathbb{R}^7.$$

A maneira de se aprender θ é familiar: iremos minimizar a soma das diferenças ao quadrado entre os valores observados da variável resposta (y_i) e os valores previstos pela rede neural ($\hat{y}_i$).

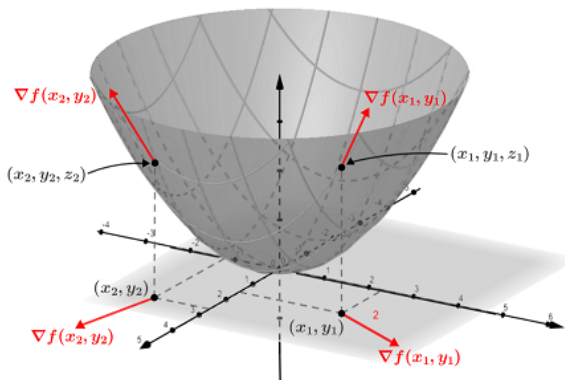
Ou seja, iremos minimizar a *função de perda quadrática*

$$L(\theta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

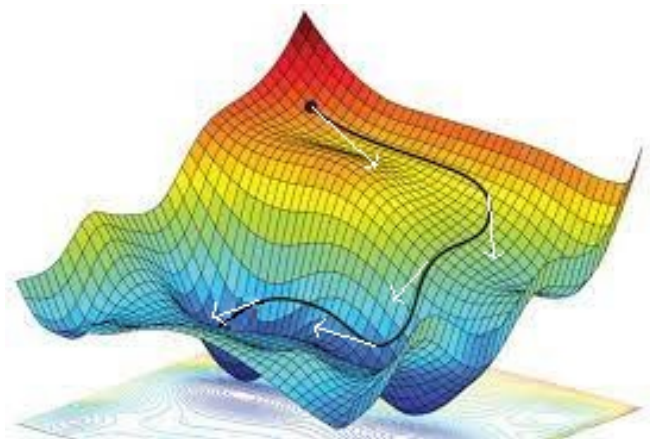
Para minimizar L , utilizaremos um método de gradiente descendente, calculando o vetor gradiente

$$\nabla L = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b_2}, \frac{\partial L}{\partial w_3}, \frac{\partial L}{\partial w_4}, \frac{\partial L}{\partial b_3} \right).$$

Lembre-se que o vetor gradiente ∇f aponta na direção de maior crescimento da função em um certo ponto.



Seguindo $-\nabla f$, chegaremos a uma ponto de mínimo (que em geral não será global).



O algoritmo de otimização via gradiente descendente, que utiliza a regra da cadeia, é denominado *backpropagation*.

Note que $\nabla L = \sum_{i=1}^n \nabla L_i$, em que $L_i(\theta) = (y_i - \hat{y}_i)^2$, de maneira que precisamos calcular as seguintes derivadas parciais.

$$\frac{\partial L_i}{\partial w_3} = \frac{\partial L_i}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial w_3} = -2(y_i - \hat{y}_i)z_1$$

$$\frac{\partial L_i}{\partial w_4} = \frac{\partial L_i}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial w_4} = -2(y_i - \hat{y}_i)z_2$$

$$\frac{\partial L_i}{\partial b_3} = \frac{\partial L_i}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial b_3} = -2(y_i - \hat{y}_i)$$

$$\frac{\partial L_i}{\partial w_1} = \frac{\partial L_i}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial z_1} \frac{\partial z_1}{\partial w_1} = -2(y_i - \hat{y}_i)w_3\sigma'(w_1x_i + b_1)x_i$$

$$\frac{\partial L_i}{\partial b_1} = \frac{\partial L_i}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial z_1} \frac{\partial z_1}{\partial b_1} = -2(y_i - \hat{y}_i)w_3\sigma'(w_1x_i + b_1)$$

$$\frac{\partial L_i}{\partial w_2} = \frac{\partial L_i}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial z_2} \frac{\partial z_2}{\partial w_2} = -2(y_i - \hat{y}_i)w_4\sigma'(w_2x_i + b_2)x_i$$

$$\frac{\partial L_i}{\partial b_2} = \frac{\partial L_i}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial z_2} \frac{\partial z_2}{\partial b_2} = -2(y_i - \hat{y}_i)w_4\sigma'(w_2x_i + b_2)$$

A implementação mais simples do algoritmo de *backpropagation* consiste em tomarmos uma observação de treinamento (x_i, y_i) por vez e seguirmos a direção oposta ao vetor gradiente da função de perda, levando em conta apenas a parcela L_i , para cada $i = 1, \dots, n$ (este processo é conhecido na literatura como *on-line learning*).

Percorreremos toda a amostra de treinamento por um certo número de *épocas* (*epochs*).

$$\delta \leftarrow 0.01$$

$$t \leftarrow 1$$

$$\theta^{(t)} \leftarrow \text{random init}$$

for epoch = 1 **to** N :

for $i = 1$ **to** n :

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \delta \nabla L_i(\theta^{(t)})$$

$$t \leftarrow t + 1$$

O valor da taxa de aprendizagem δ é escolhido empiricamente: δ muito pequeno nos fará caminhar muito lentamente pelo domínio da função da perda, enquanto δ muito grande fará com que “passemos batido” pelo ponto de mínimo desejado.

Veja na implementação do código R disponível no Blackboard que ao atualizarmos os valores das derivadas parciais é como se estivéssemos “propagando o erro para trás” na rede neural; daí o nome *backpropagation*.

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California,
San Diego, La Jolla, California 92093, USA

† Department of Computer Science, Carnegie-Mellon University,
Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure¹.

Esta primeira versão *on-line* do algoritmo de *backpropagation* acaba convergindo de maneira muito lenta quando temos uma rede neural com uma arquitetura mais complexa, envolvendo mais camadas interiores, com um número maior de neurônios. A solução encontrada para esta dificuldade consiste na construção de “mini lotes” (*mini-batches*) aleatórios dos dados, de acordo com o seguinte procedimento.

1. Embaralhamos os dados de treinamento.
2. Tomamos mini lotes de, por exemplo, 32 observações.
3. Fazemos *backpropagation* com os gradientes dos mini lotes até utilizarmos todos os dados de treinamento.
4. Os passos (1-2-3) formam uma época (*epoch*) da otimização.
5. Repetimos o processo por, digamos, 10 épocas.

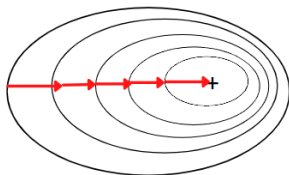
for $\ell = 1$ **to** L :

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \delta \sum_{i \in \text{Batch}_\ell} \nabla L_i(\theta^{(t)})$$

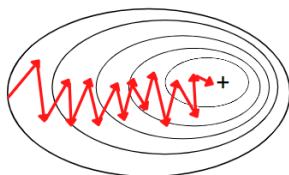
Veja os detalhes da implementação no código R.

Este é um exemplo de um *gradiente descendente estocástico* (*stochastic gradient descent* ou SGD).

Gradient Descent



Stochastic Gradient Descent



Um benefício deste processo é que a irregularidade da “descida” pelo gradiente estocástico, devido à aleatorização dos mini lotes, acaba por evitar que fiquemos presos em mínimos locais “ruins”.

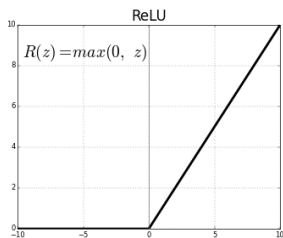
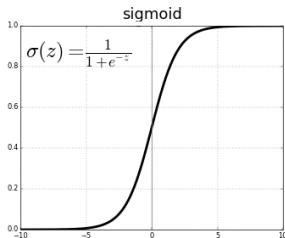
Os fundamentos teóricos gerais da ideia de *otimização estocástica* surgiram no importante trabalho de Robins e Monro em 1951.

Uma questão delicada no processo de treinamento de uma rede neural é: quando devemos parar a otimização? Em geral, se o processo for muito longo, a rede neural tende a sobreajustar os dados de treinamento, levando a uma má capacidade de generalização e a um erro de teste inadequado.

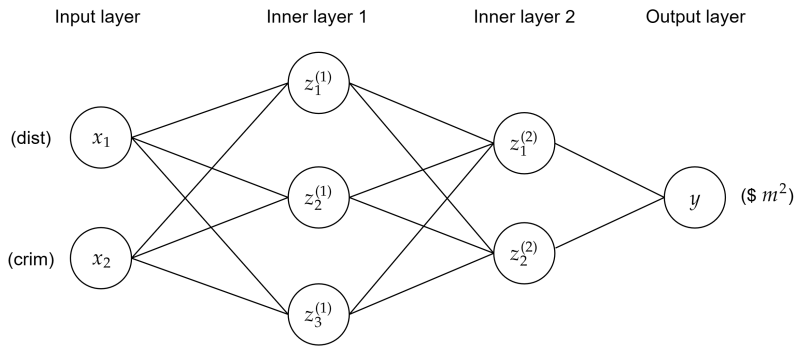
A solução utilizada na prática é, a cada época, calcular o erro em um conjunto de validação em separado. Quando este erro para de “cair”, interrompemos o processo. Este procedimento é denominado *early stopping* na literatura.

O *dropout* é um mecanismo recente que foi proposto como uma proteção ao sobreajuste da rede neural. Essencialmente, a cada iteração do processo de treinamento, cada neurônio tem uma probabilidade p de não ser incluído (o que significa que seus pesos não serão atualizados). Uma interpretação possível é que o *dropout* estaria fazendo uma “média” sobre uma família de arquiteturas distintas (uma forma primitiva de *model averaging*).

Nas aplicações modernas de maior sucesso preditivo, a função de ativação logística utilizada em nosso exemplo foi abandonada em favor da função de ativação ReLU (*Rectified Linear Unit*), definida por $\text{ReLU}(z) = \max\{0, z\}$.



O nome “ReLU” vem da eletrônica, pois esta função descreve exatamente o comportamento de um diodo, que só deixa “passar” a parte positiva da corrente em um circuito retificador.

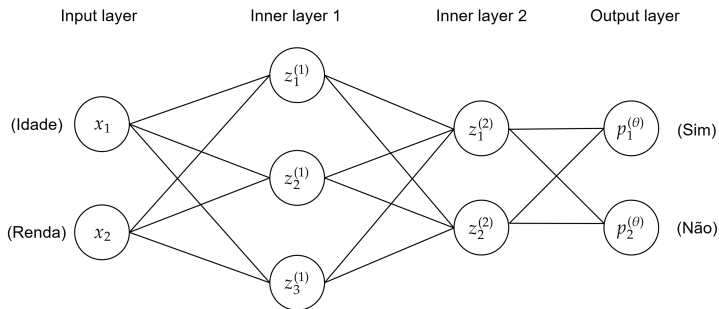


$$z_i^{(1)} = g_i^{(1)} \left(\sum_{j=1}^2 w_{ij}^{(1)} x_j + b_i^{(1)} \right) \quad i = 1, 2, 3$$

$$z_i^{(2)} = g_i^{(2)} \left(\sum_{j=1}^3 w_{ij}^{(2)} z_j^{(1)} + b_i^{(2)} \right) \quad i = 1, 2$$

$$y = \sum_{j=1}^2 w_j^{(3)} z_j^{(2)} + b^{(3)}$$

Em um problema de classificação temos uma mudança na camada de saída da rede neural, que agora passa a dar as “probabilidades” do dado ser de uma entre diversas classes. No exemplo usual em que as variáveis preditoras são a idade e a renda do cliente, e a variável resposta é se ele utiliza ou não utiliza demasiadamente o crédito automático, teríamos uma arquitetura do seguinte tipo.



$$z_i^{(1)} = g_i^{(1)} \left(\sum_{j=1}^2 w_{ij}^{(1)} x_j + b_i^{(1)} \right) \quad i = 1, 2, 3$$

$$z_i^{(2)} = g_i^{(2)} \left(\sum_{j=1}^3 w_{ij}^{(2)} z_j^{(1)} + b_i^{(2)} \right) \quad i = 1, 2$$

As variáveis da camada de saída são obtidas por uma transformação denominada *softmax*, definida por

$$p_1^{(\theta)} = \frac{e^{z_1^{(2)}}}{e^{z_1^{(2)}} + e^{z_2^{(2)}}} \quad \text{e} \quad p_2^{(\theta)} = \frac{e^{z_2^{(2)}}}{e^{z_1^{(2)}} + e^{z_2^{(2)}}}.$$

Em problemas de classificação, a variável resposta dos dados de treinamento utiliza uma codificação especial denominada *one-hot encoding*. Por exemplo, em um problema com três classes (*dog*, *cat*, *horse*), um gato seria representado pelo vetor $y = (0, 1, 0)$, no qual apenas a coordenada correspondente à classe *cat* seria igual a 1 (estaria “ligada” ou “quente”/*hot*), enquanto as demais seriam iguais a 0 (estariam “desligadas” ou “frias”).

Ademais, no processo de *backpropagation* não minimizamos mais a perda quadrática dos problemas de regressão. Em classificação, a função de perda a ser minimizada é a *entropia cruzada*

$$L(\theta) = - \sum_{i=1}^n \sum_{j=1}^k y_{ij} \ln p_j^{(\theta)}(x_i),$$

o que equivale a maximizar a função de verossimilhança de um modelo multinomial da forma

$$\prod_{i=1}^n \prod_{j=1}^k \left(p_j^{(\theta)}(x_i) \right)^{y_{ij}}.$$